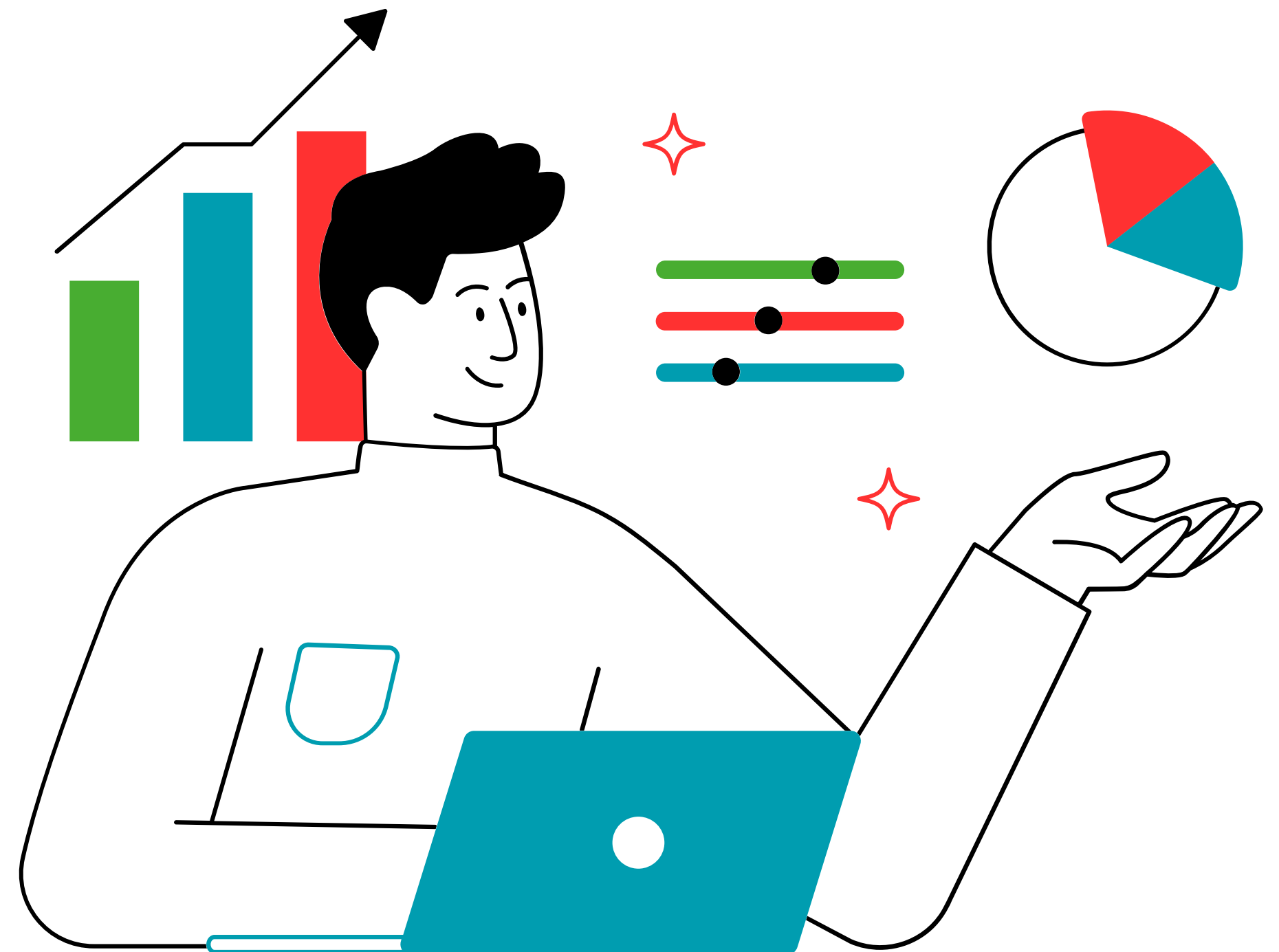


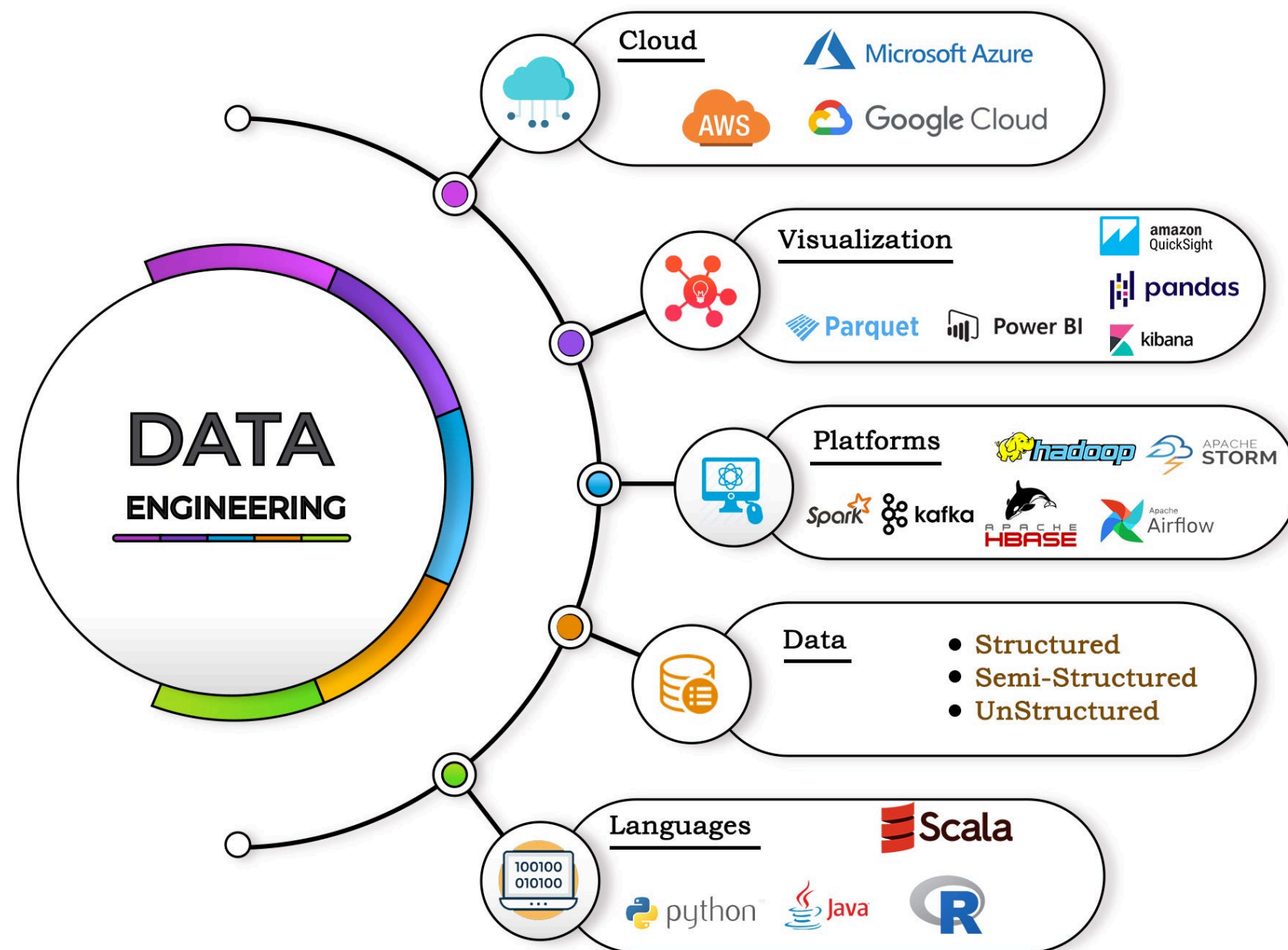
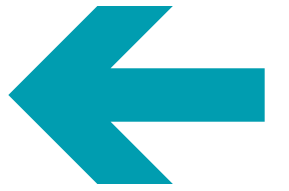
Data Engineering Study Guide.

2026



www.brightlearn.co.za

What is a Data Engineering ?



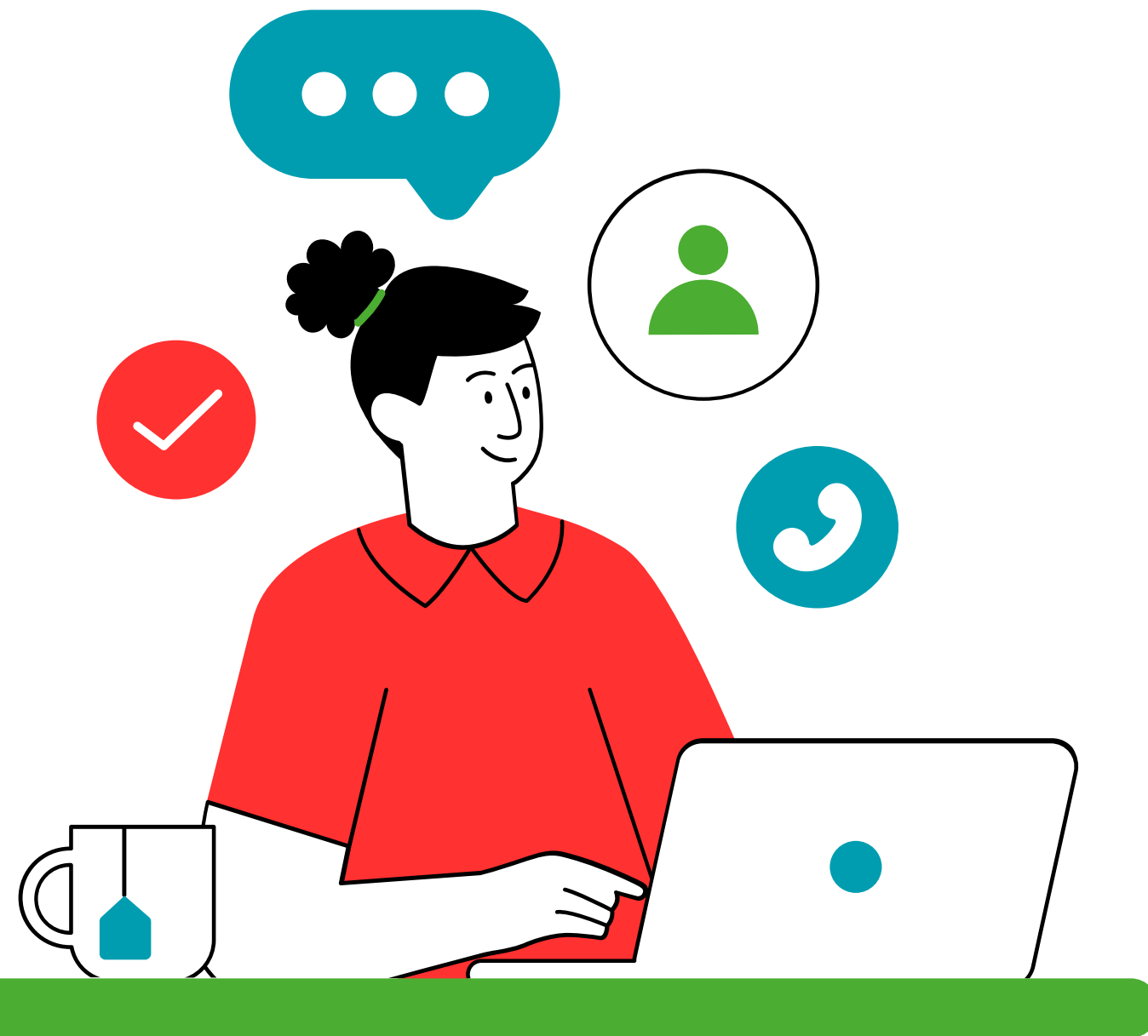
A **Data Engineer** is a professional responsible for designing, building, and maintaining the systems that collect, process, and store data for organizations. Their work ensures that data is reliable, accessible, and ready for analysis and decision-making.

Data Engineers work with raw data from multiple sources, creating ETL pipelines to extract, transform, and load data into databases and data warehouses. They ensure data quality, consistency, and performance using tools such as SQL, Python, SSIS, and cloud platforms like Azure.

Data Engineers also focus on data modelling, database optimization, and automation, enabling analysts and business teams to access clean, well-structured data. They collaborate closely with data analysts, data scientists, and other teams to support analytics, reporting, and scalable data solutions that drive business performance.

-
- A vibrant, cartoon-style illustration of a person with dark hair, wearing a red hoodie and a red beanie, sitting at a desk and working on a laptop. The person is smiling and looking towards the camera. The laptop screen displays a simplified version of the network diagram seen on the larger monitor. The large monitor in the background shows a complex network architecture with multiple blue and green server nodes connected by lines, and a small inset window showing a bar chart. The background is light blue and filled with various floating icons: a cloud with a green house, a large blue gear, a smaller green gear, a stack of blue disks, a cloud with a bar chart, a stack of green disks, a green gear, a donut chart, and a potted plant with green leaves in a blue pot. The overall style is modern and tech-oriented.

Essential Skills for Data Engineer

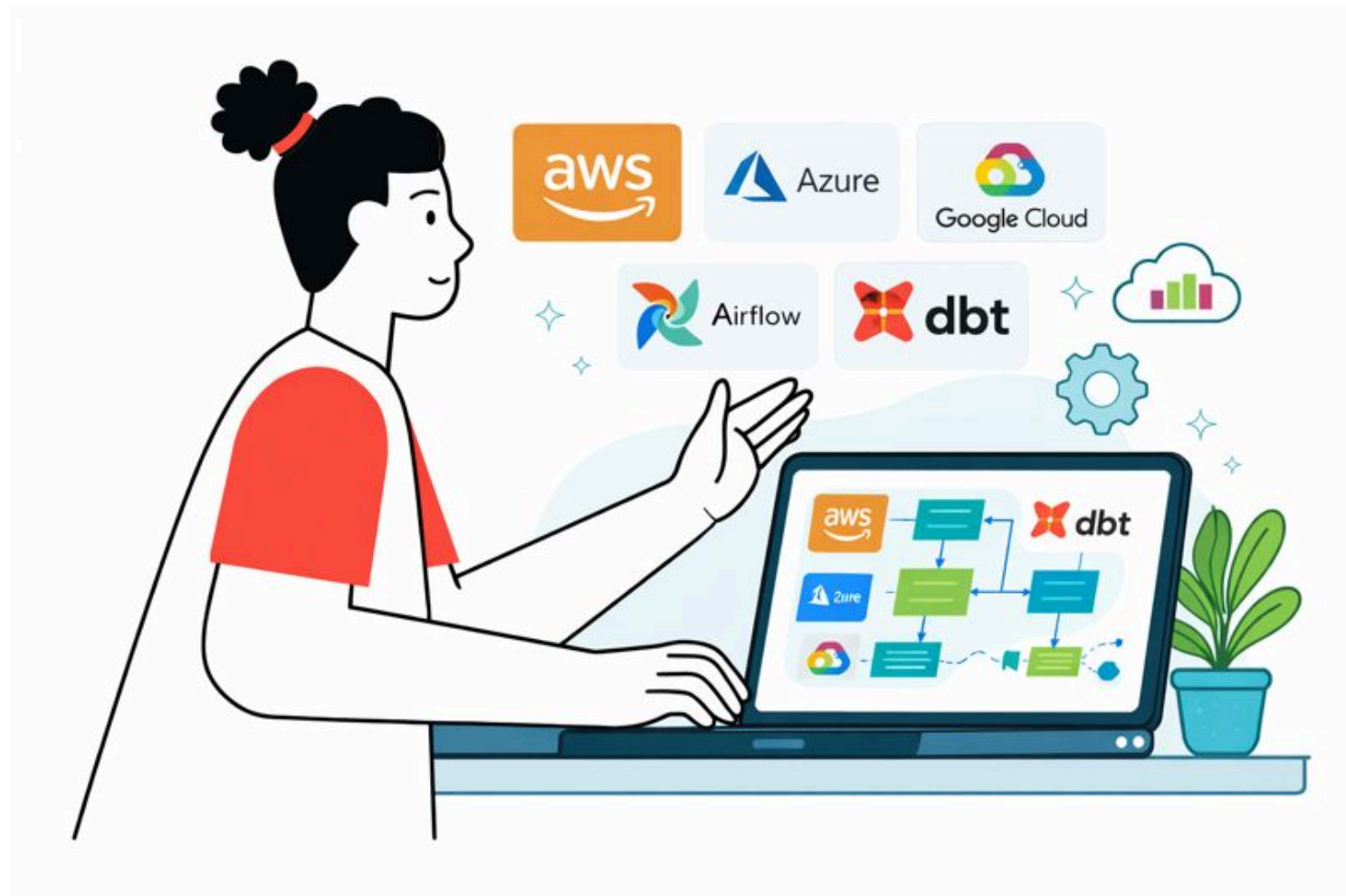


Technical Skills

- Data Manipulation & Cleaning
- Statistical Analysis
- Data Visualization
- Programming
- Database Management
- Data Integration and ETL
- Tools
- Version Control
- Business Intelligence Tools

Soft Skills

- Problem-Solving
- Attention to Detail
- Communication Skills
- Collaboration and Teamwork
- Time Management
- Critical Thinking
- Adaptability
- Curiosity and Learning Mindset
- Presentation Skills
- Ethical Thinking and Integrity



Tools & Technologies

Understanding Data Engineering Tools & Technologies

You'll learn how to build reliable data pipelines that turn raw data into clean, usable data. Using SSIS and SQL Server, you'll design ETL processes and understand how data is stored for accurate analysis. You'll also use GitHub for version control and Azure Cloud to deploy scalable data solutions that support business decision-making.

You'll develop strong skills in SQL and Python for querying, transforming, and automating data workflows, along with data modelling techniques to design efficient, well-structured databases that support reliable analytics and reporting. This hands-on approach prepares you for real-world data engineering roles.

Data Cleaning:

- Microsoft Excel
- Google Sheets
- Structure Query Language (SQL)

Data Visualization:

- Microsoft Excel
- Google Sheets
- PowerBI

Data Presentation:

- Microsoft Power Point
- Google Slides
- Canva

Project Management

- Miro
- Canva
- Figma
- Microsoft Excel
- Google Sheets

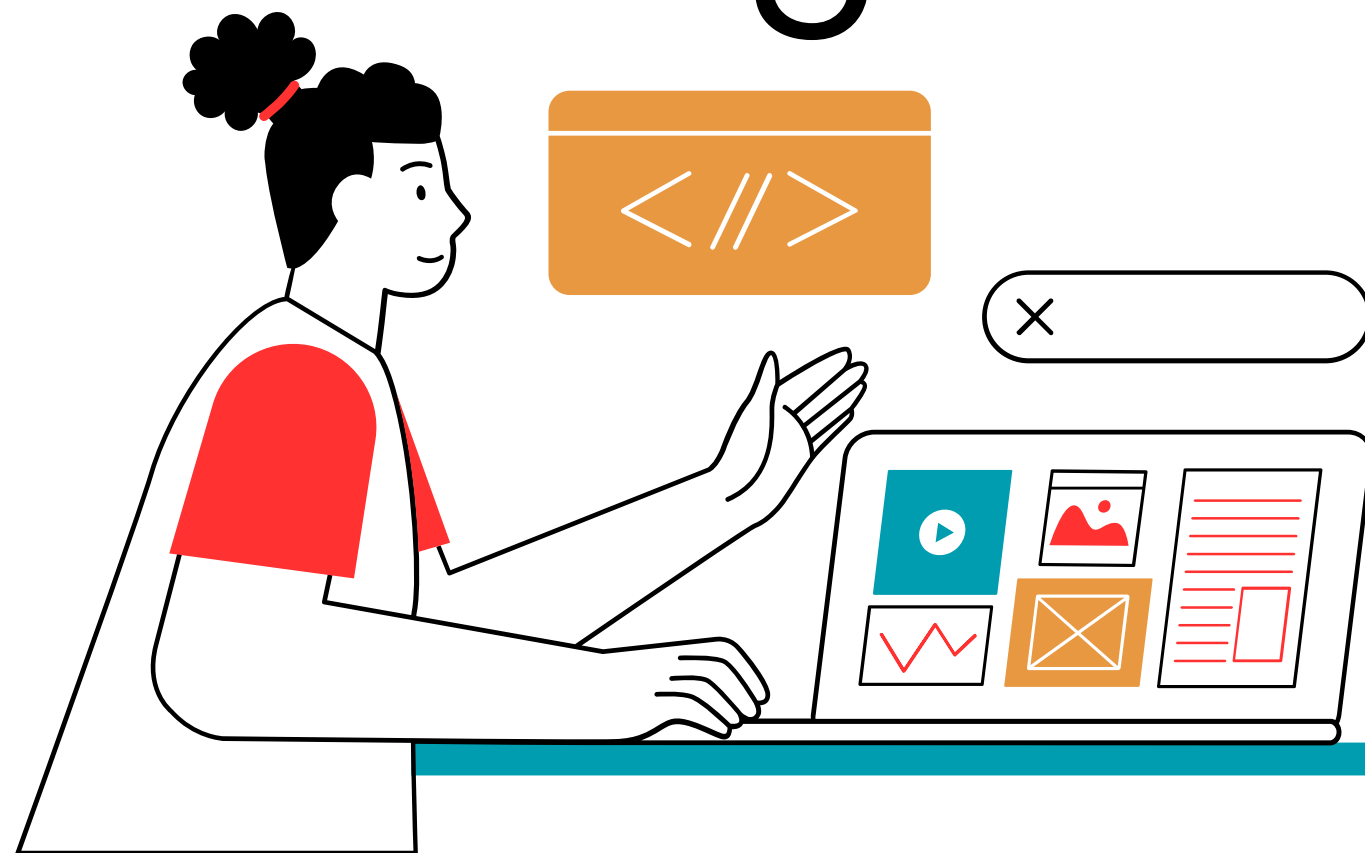
Databases:

- Databricks
- Snowflake
- Microsoft SQL Server
- My SQL Workbench

Version Control:

- GitHub

Access Course Tools & Technologies



Platform	Purpose	Platform Link
Databricks	Coding SQL	https://www.databricks.com/
Snowflake	Coding in SQL and building Databases	https://signup.snowflake.com/
BigQuery (Google Cloud)	Coding in SQL	https://console.cloud.google.com/bigquery
GitHub	Version control, sharing projects and building your data analytics portfolio.	https://github.com/
Canva	Building presentations	https://www.canva.com/templates
Miro	Project planning	https://miro.com/login/
Figma	Project planning	https://www.figma.com/design/UPUjGXCpd4S0Ok9ZgCmHEZ/Untitled?node-id=1-19&t=cmvFEdKGb4ZjCmgT-0
Looker Studio	Creating visual dashboards	https://lookerstudio.google.com/

Data Engineering Tools .



Used to design, build, and manage data engineering solutions, especially SSIS packages. Data engineers use Visual Studio to create ETL pipelines that move and transform data from source systems into databases and data warehouses.



Used to test and explore APIs that provide data from external systems. It helps data engineers understand API responses, authentication, and data formats before integrating API data into ETL pipelines.



Used for data transformation, automation, and processing. Python allows data engineers to clean and prepare data, automate repetitive tasks, and handle complex data logic that cannot easily be done with SQL alone.



Used for version control and team collaboration. GitHub stores code securely, tracks changes over time, and allows teams to work together on data engineering projects using professional development practices.



Provides the cloud environment where data pipelines, databases, and storage systems are deployed. Azure enables scalable, secure, and reliable data engineering solutions that can grow with business needs.



Visual Studio Code

A lightweight and flexible code editor used to write and manage Python scripts, SQL queries, and configuration files. It integrates seamlessly with GitHub and cloud tools, making development more efficient.



databricks

Used for large-scale data processing and analytics. Databricks helps data engineers process big datasets efficiently using distributed computing and supports advanced data transformation workflows.



Used to store, manage, and query data. SQL is the core language for extracting, transforming, and loading data, while SQL Server provides a reliable platform for databases and data warehouses.

Data Engineering Program Layout

Course Overview

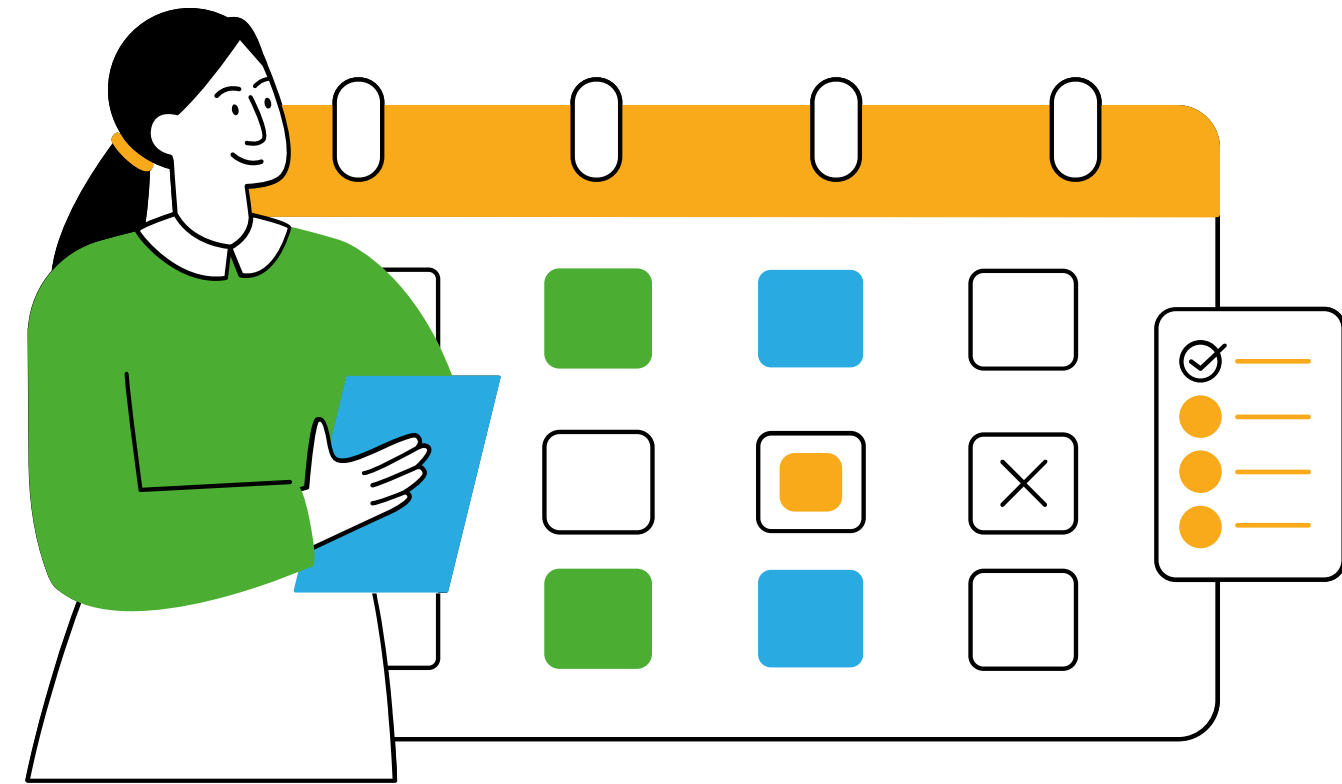
- Duration : 6 Months
- Level : Beginners
- Course Delivery : Online
- Tools Needed: Laptop & WiFi
- Additional Tools: Notebook & Pen
- Price: R550 per month
- Class Schedule: Monday – Thursday
- Class Time: 19:00 – 20:00
- Consultations: Saturday & Sunday

Course Curriculum

- **Module 1:** Tools Setup & SQL Foundations
- **Module 2:** Intermediate SQL & Automation
- **Module 3:** SSIS Foundations
- **Module 4:** Advanced SSIS
- **Module 5:** Python & GitHub
- **Module 6:** Capstone Project

Skills You Will Gain

- Data analysis and interpretation
- Data cleaning, wrangling, and preparation using SQL
- SQL for data manipulation and data extraction
- Building reliable ETL processes and data pipelines
- Working with databases and data warehouses
- Version control and collaboration using GitHub
- Deploying data solutions in the Azure Cloud
- Developing a professional capstone project to demonstrate your data engineering skills



- **Module 1:** Tools Setup & SQL Foundations
- **Module 2:** Intermediate SQL & Automation
- **Module 3:** SSIS Foundations
- **Module 4:** Advanced SSIS
- **Module 5:** Python & GitHub
- **Module 6:** Capstone Project

Data Engineering RoadMap.

Introduction to Databases

Rational Databases (OLAP & OLTP), Data warehousing, Cloud Data systems, Data Management Systems



SQL Fundamentals

Learn SQL Basic syntax, including Aggregate functions, arithmetic operator, logical operators, etc



SSIS Foundations

Connectors ,Data Transformations
Data Cleansing and Validation



Advanced SSIS & Cloud

Performance Tuning and Optimization,
Deployment to SSISDB,
Workflow Automation with SQL Server
Agent, Azure

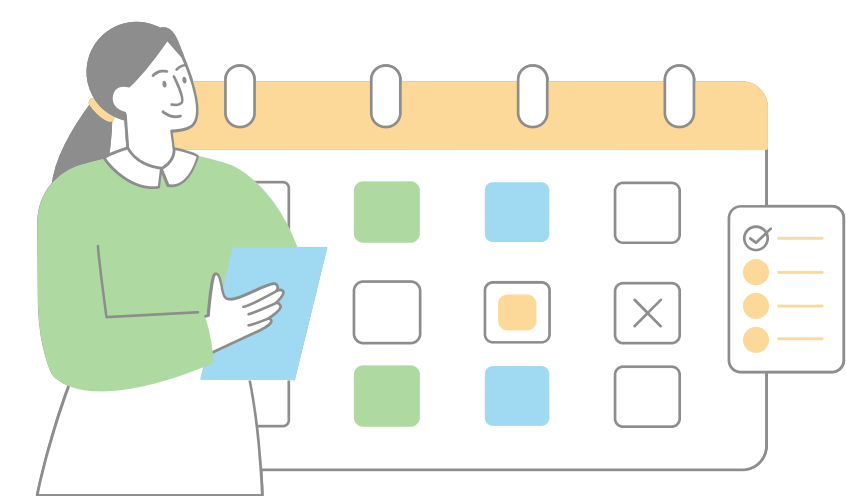
Python & Github

SQL Integration and Database Connectivity,
Version Control, Branching, Merging, and Pull
Requests



Capstone Project

Data Modeling ,
End-to-End ETL Pipeline Development
Testing, Optimization, Final Documentation
Project Presentation



Detailed Course Curriculum :

Month 1: Tools Setup & SQL Foundations

- Week 1: Tools Setup – SQL Server, SSMS, SSIS, Python, VS Code
- Week 2: SQL Basics – SELECT, WHERE, ORDER BY
- Week 3: Aggregations – COUNT, SUM, AVG, GROUP BY
- Week 4: SQL Joins – INNER, LEFT, RIGHT, FULL

Month 2: Intermediate SQL & Automation

- Week 5: Subqueries and Common Table Expressions (CTEs)
- Week 6: Window Functions
- Week 7: Stored Procedures and User-Defined Functions
- Week 8: SQL Automation using SQL Server Agent

Month 3: SSIS Foundations

- Week 9: Introduction to SSIS and ETL Concepts
- Week 10: Data Sources and Connectors
- Week 11: Data Transformations
- Week 12: Data Cleansing and Validation

Month 4: Advanced SSIS

- Week 13: Error Handling and Logging
- Week 14: Performance Tuning and Optimization
- Week 15: Deployment to SSISDB
- Week 16: Workflow Automation with SQL Server Agent

Month 5: Python & GitHub

- Week 17: Python Fundamentals for ETL
- Week 18: File Handling and Data Processing in Python
- Week 19: Python–SQL Integration and Database Connectivity
- Week 20: Git and GitHub – Version Control, Branching, Merging, and Pull Requests

Month 6: Capstone Project

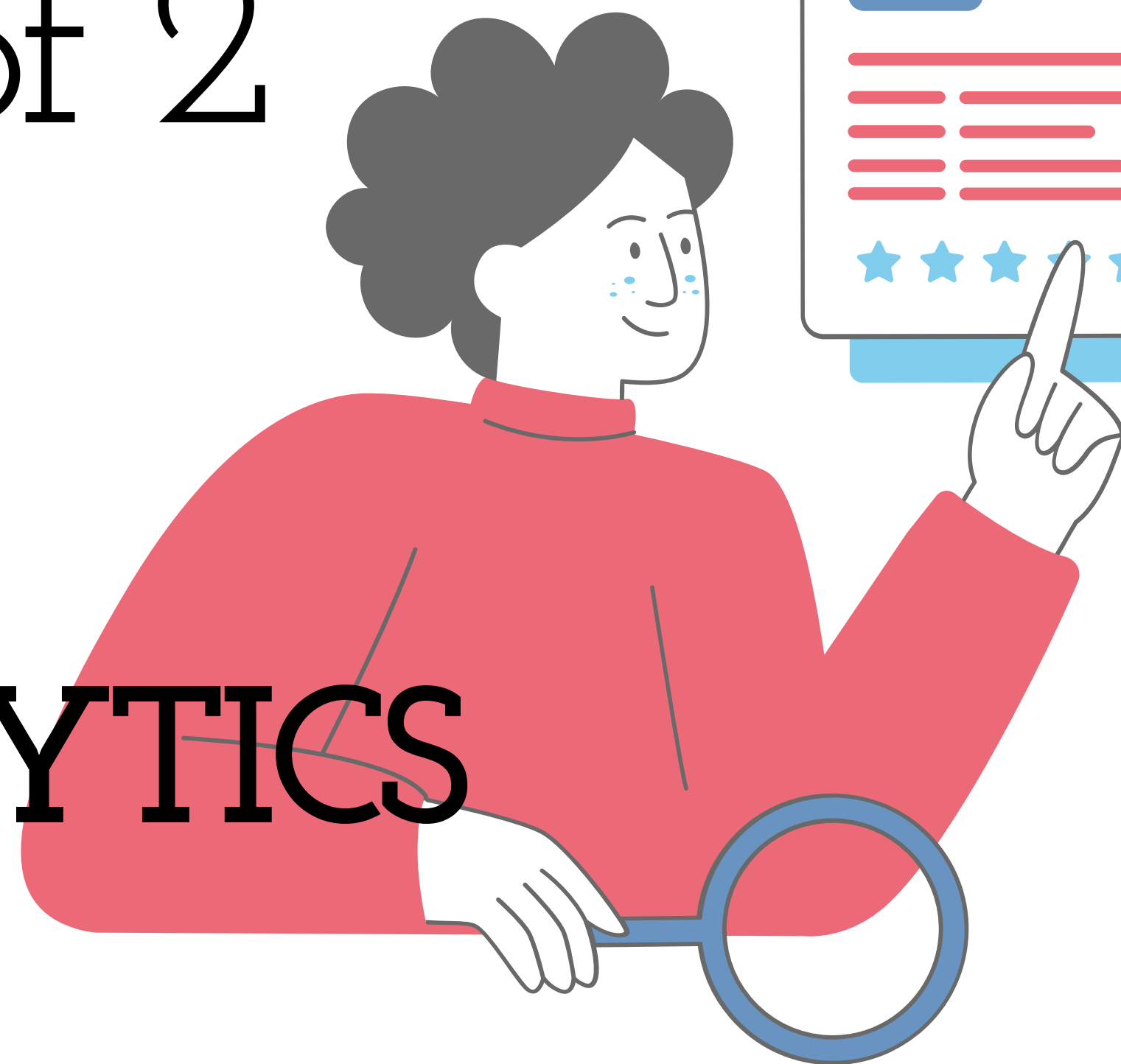
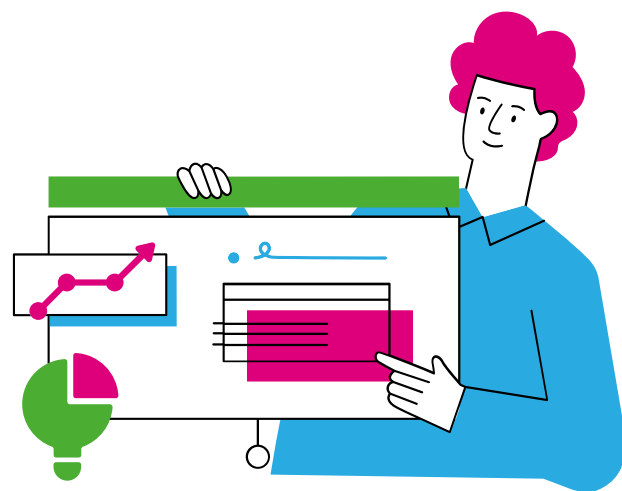
- Week 21: Data Modeling and Project Planning
- Week 22: End-to-End ETL Pipeline Development
- Week 23: Testing, Optimization, and Performance Review
- Week 24: Final Documentation and Project Presentation



Part 1 of 2

Introduction.

DATA ANALYTICS



Introduction to SQL

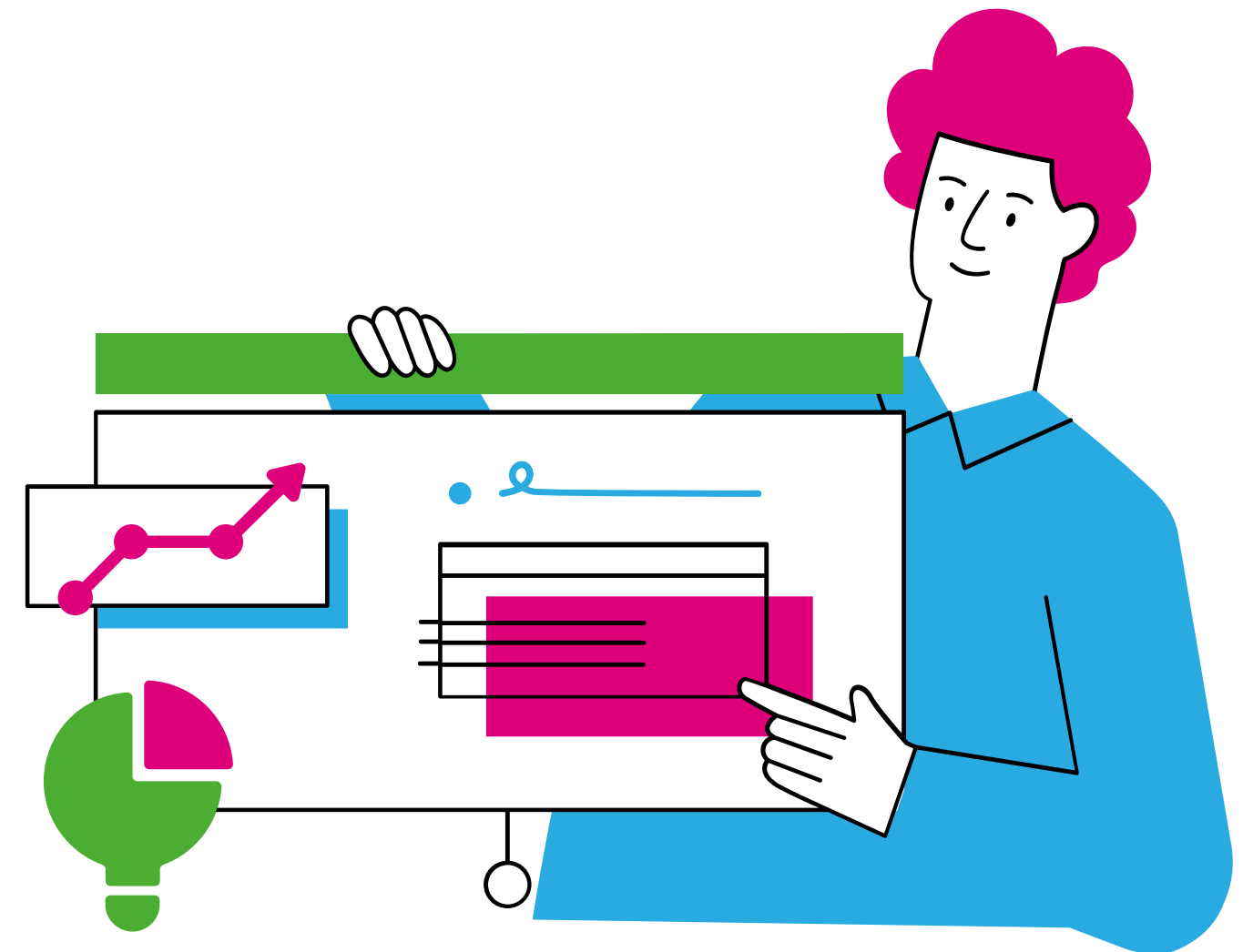
What is SQL?

SQL stands for Structured Query Language. It is the standard language used to store, retrieve, and manipulate data in relational databases. Whether you're analyzing sales data, customer information, or performance metrics, SQL is an essential tool in a data analyst's toolkit.

What can SQL do?

SQL With SQL, you can:

- Execute queries against a database
- Retrieve specific data from one or more tables
- Insert new records into a database
- Update existing records
- Delete unwanted records
- Create new databases and tables
- Create and manage stored procedures
- Create database views
- Set permissions on tables, procedures, and views



Introduction to SQL

What is Relational Database?

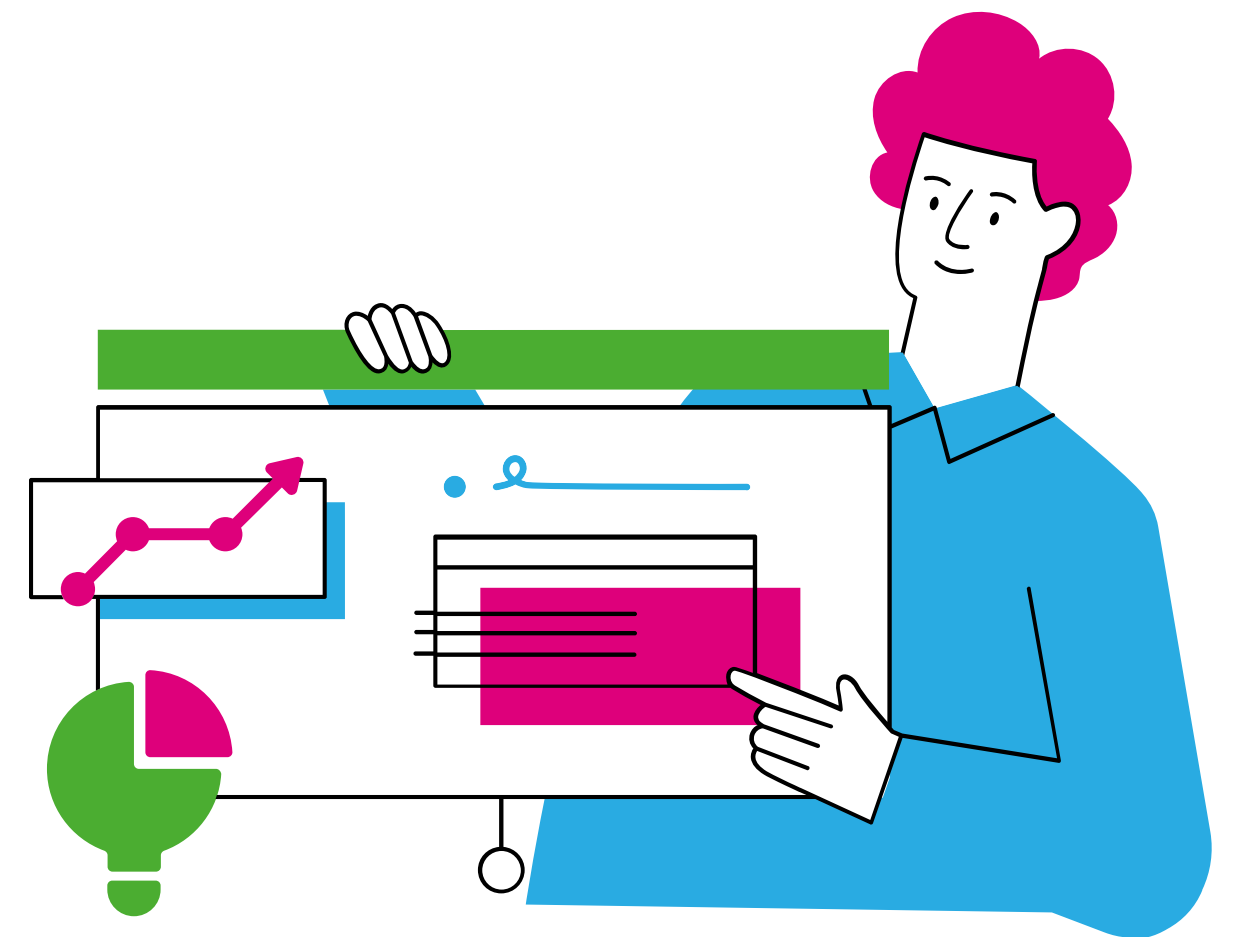
A relational database stores data in tables (rows and columns)

What is Relational Database Management System (RDBMS)?

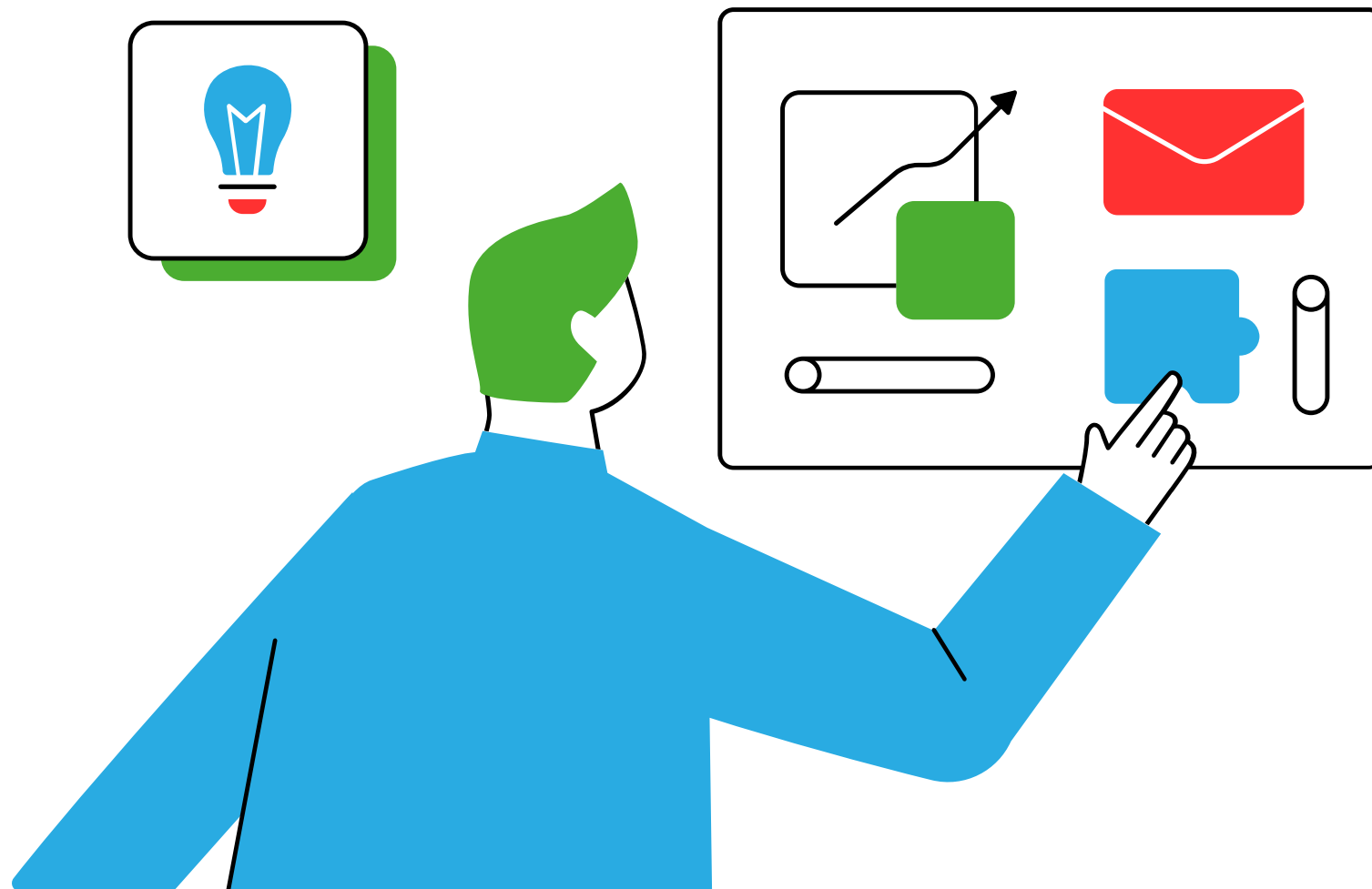
An RDBMS is the software/platform that creates, stores, secures, and manages those relational tables and lets you run SQL reliably

Relational Database Management System (RDBMS)?

- **Organizes data reliably:** Stores data in structured tables with rules (schemas) so it stays clean and consistent.
- **Enables fast SQL querying:** Lets you retrieve, filter, join, and analyze data efficiently using SQL.
- **Protects and controls access:** Manages users, roles, and permissions so the right people see the right data.
- **Maintains data integrity:** Uses constraints and transactions to prevent duplicates, invalid values, and partial updates.
- **Supports scalability and performance:** Provides indexing/optimization and managed storage/compute to handle growing data and users.



SQL Content Breakdown



SQL Fundamentals

This section will introduce basic SQL Syntax

1.1 SELECT Statements

- SELECT
- SELECT DISTINCT
- FROM
- AS (Aliases)

1.2 Filtering and Sorting

- WHERE Statement
- ORDER BY Clause
- SQL COMMENTS

1.3 SQL Operators

- Arithmetic Operators
- Bitwise Operators
- Comparison Operators
- Compound Operators
- Logical Operators

1.4 SQL Aggregate Functions s Filtering Grouped Data

- SUM()
- AVG()
- COUNT()
- MIN() and MAX()
- GROUP BY Clause
- HAVING Clause

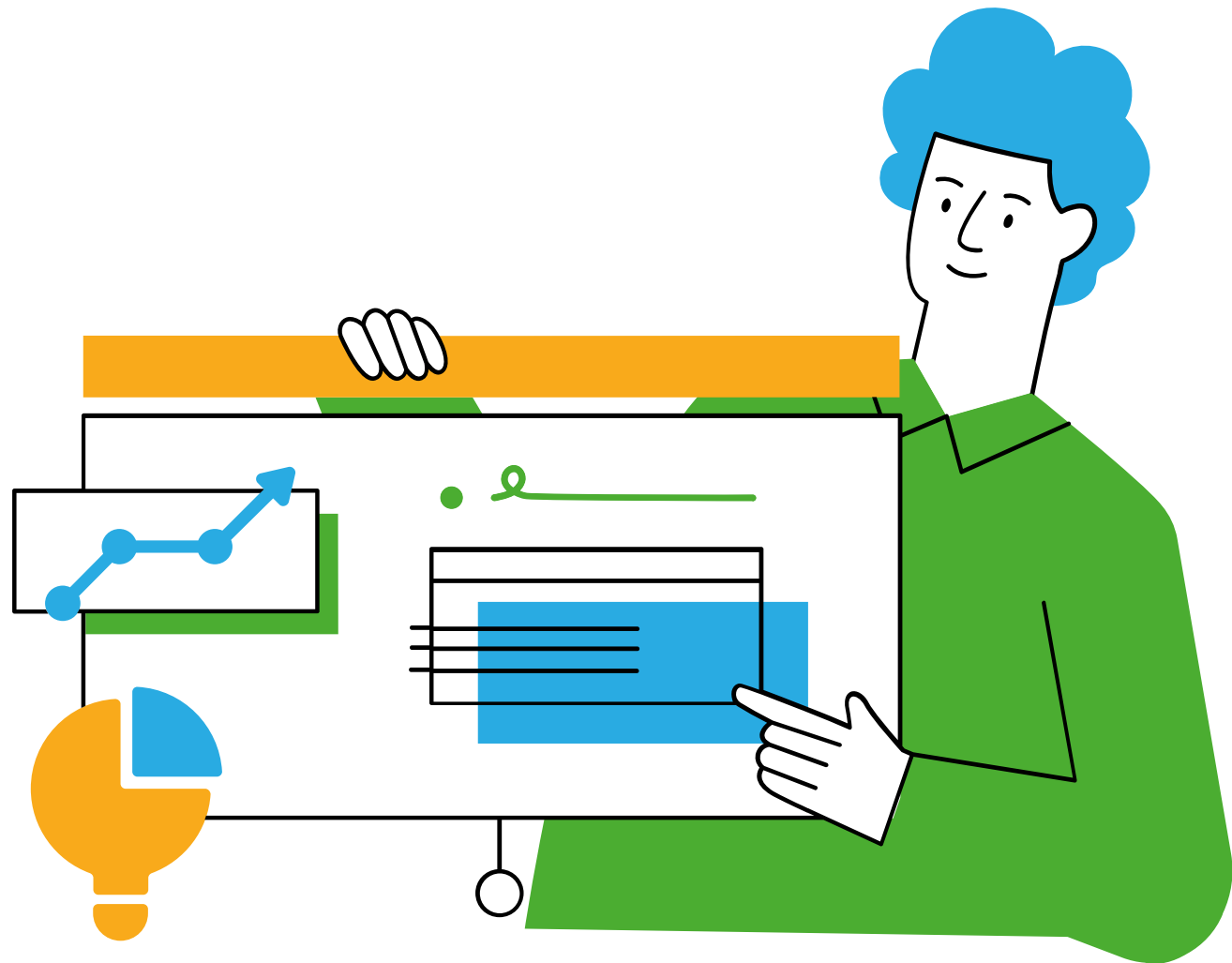
1.5 Combining Tables (Joins and Unions)

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- SELF JOIN
- CROSS JOIN
- UNION Operator

1.6 Conditional Logic

- CASE Statements

SQL Content Breakdown



Advanced SQL

This section will introduce Advanced SQL Syntax

2.1 Wildcards and Patterns

- SQL Wildcards (LIKE, %, _)

2.2 Filtering and Sorting

- NULL Functions
- DATE Functions

2.3 SQL Operators

- CREATE DATABASE
- DROP DATABASE
- BACKUP DATABASE

2.4 Table Operations

- CREATE TABLE
- DROP TABLE
- ALTER TABLE

2.7 Indexes

- CREATE INDEX
- DROP INDEX

2.5 Data Manipulation

- INSERT INTO
- SELECT INTO
- INSERT INTO SELECT
- UPDATE
- DELETE

2.6 Constraints

- UNIQUE Constraint
- CHECK Constraint
- DEFAULT Constraint

2.6 Constraints

- UNIQUE Constraint
- CHECK Constraint
- DEFAULT Constraint

2.8 Advanced Clauses

- EXISTS
- ALL Operator
- ANY Operator

2.9 Stored Procedures

- Creating and Using Stored Procedures

Career Path & Opportunities: Analytics

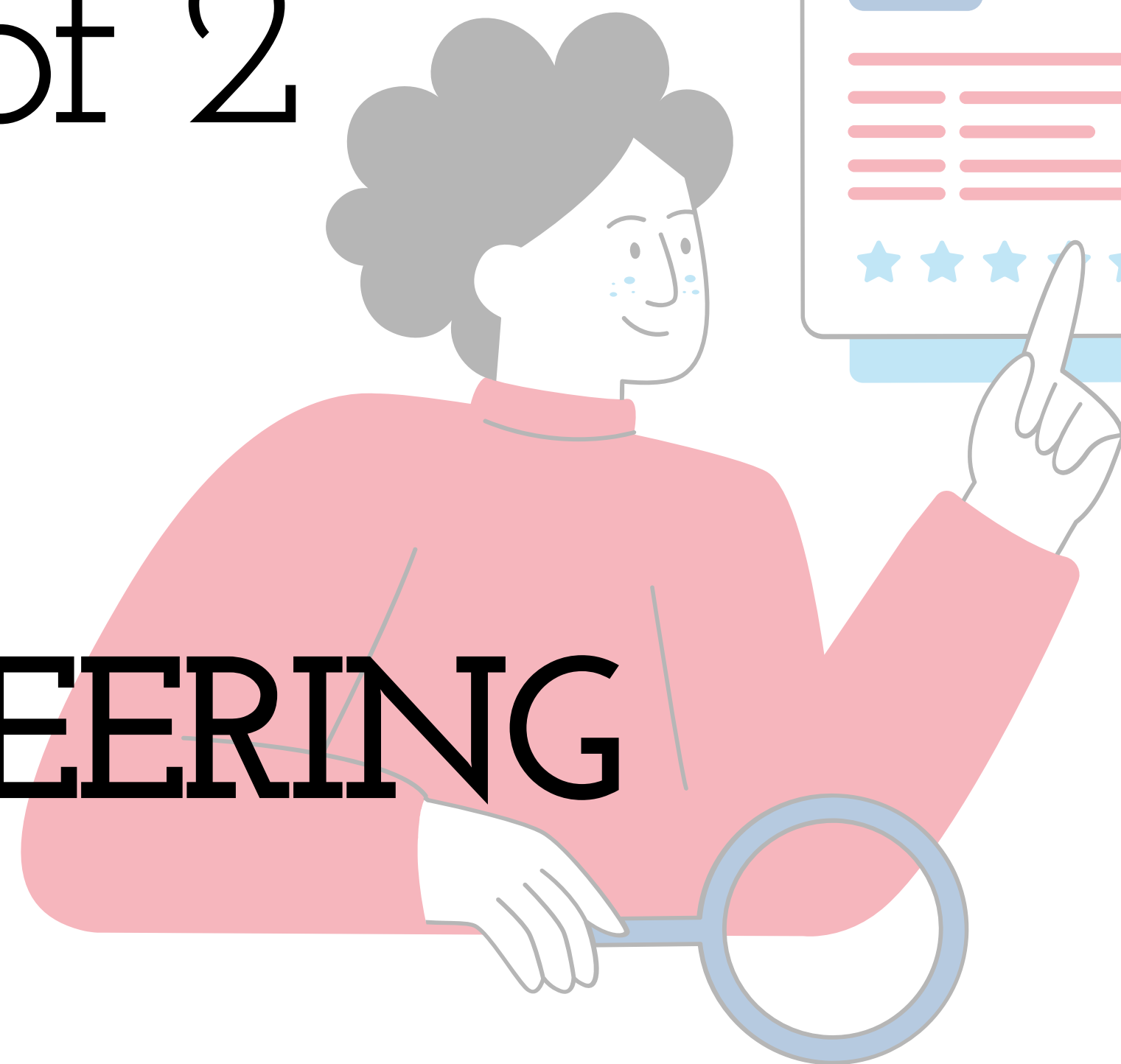
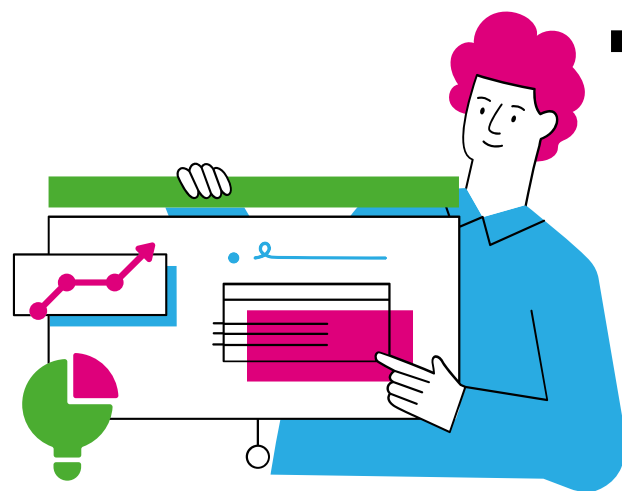
With **SQL + Excel + Power BI + data presentation**, you can qualify for roles across most industries because those skills are the “core toolkit” for turning raw data into business decisions. Here are strong career opportunities:

- **Data Analyst** – build reports/dashboards, track KPIs, explain trends and performance.
- **Business Intelligence (BI) Analyst** – design Power BI dashboards, data models, and executive reporting.
- **Reporting / MI (Management Information) Analyst** – scheduled reporting, scorecards, operational insights.
- **Product / Customer Insights Analyst** – analyze customer behavior, retention, funnels, segments, churn.
- **Marketing Analyst** – campaign performance, attribution, ROI, audience targeting insights.
- **Sales / Revenue Analyst** – pipeline, conversion, pricing analysis, sales performance dashboards.
- **Operations Analyst** – efficiency, turnaround times, cost tracking, process improvement using data.
- **Healthcare / Clinical Data Analyst** – patient flows, outcomes tracking, operational reporting (where applicable).
- **Government / Public Sector Data Analyst** – service delivery metrics, monitoring and evaluation reporting.
- **Big Data Analyst (entry path)** – if you add Databricks/Spark basics, you can analyze larger datasets.
- **Data Engineer / Analytics Engineer (junior path)** – if you add ETL + data modeling, you can build pipelines and curated datasets for BI.
- **Data Scientist (junior path)** – if you add statistics + Python + ML, you can move into predictive analytics.
- **Quantitative Analyst (limited path)** – usually needs strong statistics/programming, but your skills help as a foundation.
- **Finance Analyst / FP&A Analyst** – budgeting, forecasting, variance analysis, financial dashboards.
- **Supply Chain / Logistics Analyst** – inventory, demand trends, delivery performance, cost optimization.
- **HR / People Analytics Analyst** – hiring metrics, retention, performance, workforce planning.
- **Risk / Fraud Analyst** – detect unusual patterns, monitor risk indicators, reduce losses.

Part 2 of 2

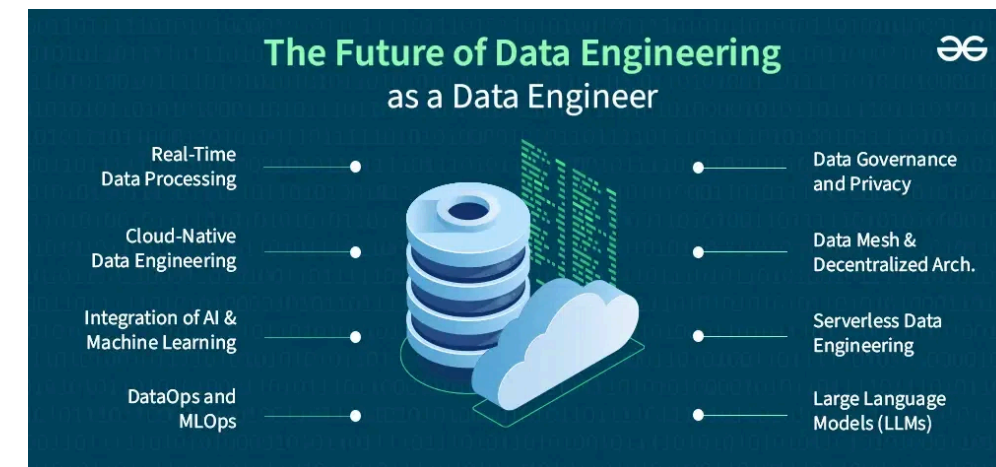
Introduction.

DATA ENGINEERING





Career Path & Opportunities : Data Engineering.

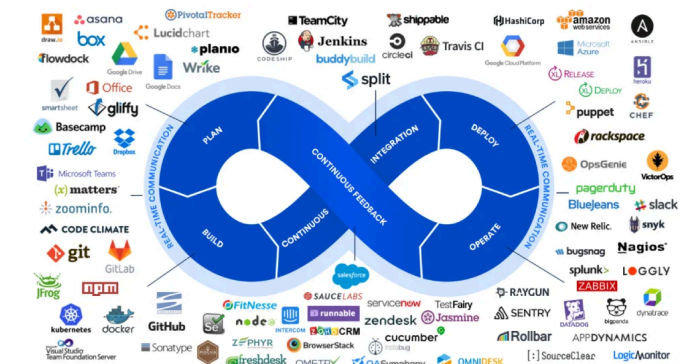


Career Path & Opportunities (Data Engineering)

With SQL + Python + ETL + Data Warehousing + Cloud + Data Modeling, you qualify for roles focused on building, scaling, and maintaining data infrastructure that powers analytics, reporting, and machine learning across industries.

Strong Career Opportunities

- **Junior Data Engineer** – build and maintain data pipelines, transform raw data into analytics-ready datasets.
- **Data Engineer** – design ETL/ELT pipelines, manage data warehouses/lakes, ensure data reliability and performance.
- **Analytics Engineer** – model data for BI tools, apply best practices (dbt, star schemas), bridge analytics and engineering.
- **Business Intelligence Engineer** – support dashboards by engineering clean, performant data layers.
- **Platform / Infrastructure Data Engineer** – manage data platforms (Databricks, Snowflake, BigQuery), optimize cost and performance.
- **Cloud Data Engineer** – build scalable pipelines using AWS, Azure, or GCP services.
- **Streaming / Real-Time Data Engineer** – process real-time data using Kafka, Spark Streaming, or Flink.
- **Machine Learning Engineer (entry path)** – supply clean, feature-ready datasets and pipelines for ML models.
- **DataOps Engineer** – monitor pipelines, automate deployments, improve data quality and reliability.
- **Big Data Engineer** – work with large-scale datasets using Spark, Hadoop, and distributed systems.
- **Software Engineer (Data-focused)** – apply engineering principles to data systems and internal platforms.
- **Finance / Risk Data Engineer** – build secure, auditable pipelines for financial and regulatory data.
- **Healthcare / Clinical Data Engineer** – manage sensitive data pipelines, compliance-focused architectures.

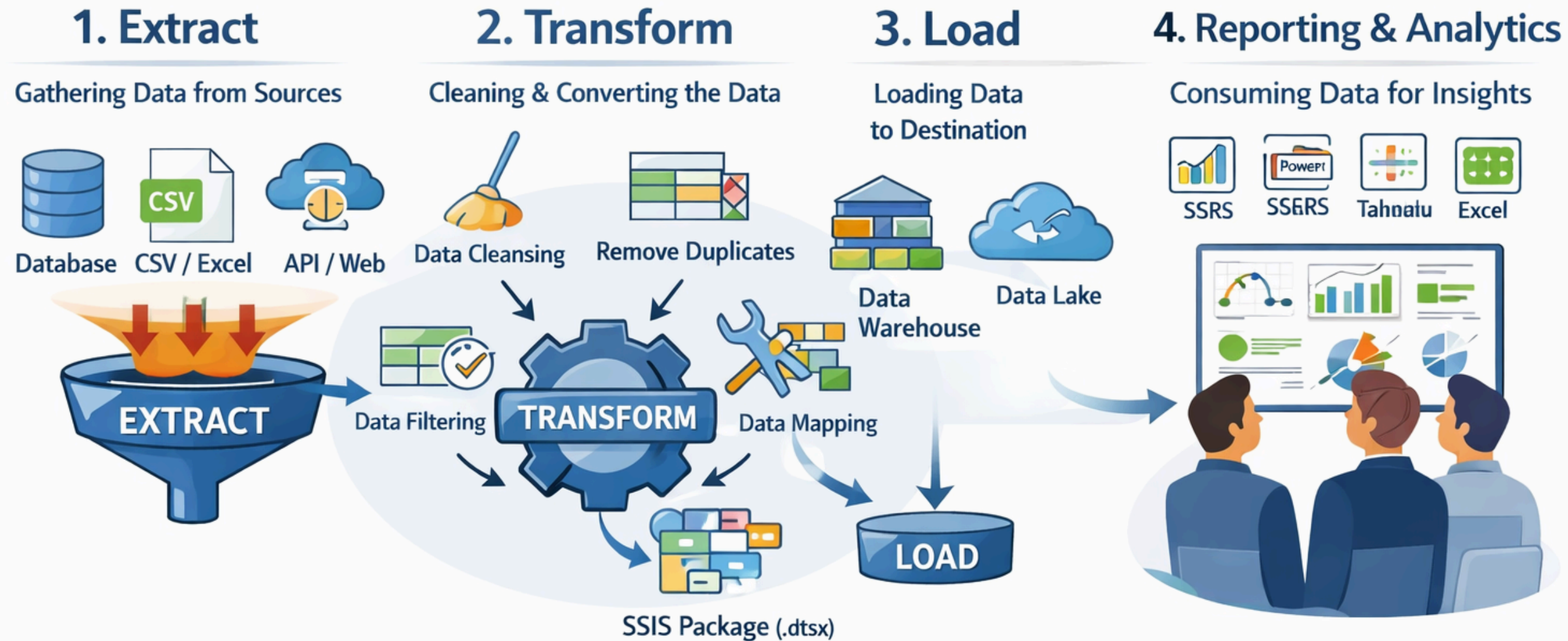


Fundamentals of SSIS in Data Engineering

SSIS Terminology for Data Engineering

SSIS Term	Description	Role in Data Engineering / ETL
SSIS Package	A collection of tasks and workflows saved as a <code>.dtsx</code> file	Represents a complete ETL pipeline
Control Flow	Defines the workflow and execution order of tasks	Orchestrates ETL logic and dependencies
Data Flow	Handles data extraction, transformation, and loading	Core component for moving and transforming data
Task	A unit of work in Control Flow (e.g., Execute SQL Task)	Performs operations such as SQL execution or file handling
Container	Groups tasks together (Sequence, For Loop, Foreach Loop)	Enables looping and logical grouping of tasks
Precedence Constraint	Controls task execution order based on success or failure	Ensures correct ETL execution logic
Connection Manager	Defines connections to data sources and destinations	Enables SSIS to connect to databases, files, and services
Source Component	Extracts data from a system	Entry point of data into the pipeline
Transformation	Modifies data within the Data Flow	Cleans, enriches, and reshapes data
Destination Component	Loads data into a target system	Final step of the ETL process

Lookup Transformation	Matches data against reference tables	Used for data enrichment and validation
Derived Column	Creates or modifies column values	Used for calculations and data standardization
Conditional Split	Routes data rows based on conditions	Supports data validation and branching logic
Variables	Store values used during package execution	Enable dynamic and reusable packages
Parameters	External inputs to SSIS packages	Support environment-specific configuration
Expressions	Dynamic logic used to modify properties at runtime	Enables flexible file paths and SQL queries
Error Output	Captures rows that fail during processing	Supports data quality and error handling
Event Handlers	Respond to runtime events (e.g., OnError)	Used for logging and notifications
Logging	Records execution details	Supports monitoring and debugging
SSIS Catalog (SSISDB)	Central repository for deployed packages	Enables execution, monitoring, and parameter management
Environment	Collection of configuration values in SSISDB	Supports dev, test, and production deployments
SQL Server Agent	Schedules and automates SSIS package execution	Enables production-grade ETL automation



ETL Process with SSIS: Extract → Transform → Load → Report

Data Engineers focus on Stages 1–3. Insights happen in Stage 4.

ETL prepares the data. Reporting consumes the data.

INTRODUCTION TO SSIS (SQL SERVER INTEGRATION SERVICES).

1. What is SSIS?

SQL Server Integration Services (SSIS) is Microsoft's ETL (Extract, Transform, Load) tool used to move, transform, and integrate data across systems. It is designed to handle large-scale data integration tasks in a reliable and automated manner.

2. Why Use SSIS?

SSIS is used to:

- Extract data from multiple sources (databases, files, APIs)
- Transform data through cleansing, validation, and enrichment
- Load data into data warehouses or analytical systems
- Automate and schedule data workflows
- Handle errors, logging, and monitoring

3. SSIS Architecture Overview

Key SSIS components include:

- Control Flow – Defines the workflow and task execution order
- Data Flow – Handles data extraction, transformation, and loading
- Connection Managers – Manage connections to data sources
- Parameters & Variables – Enable dynamic and reusable packages

4. SSIS Data Flow Components

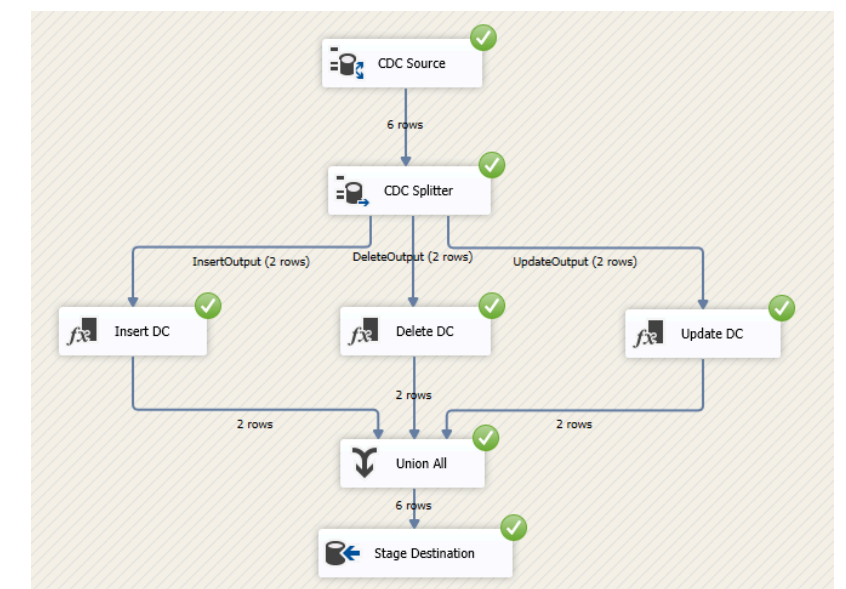
- Sources – SQL Server, Flat Files, Excel, OLE DB
- Transformations – Derived Column, Lookup, Merge, Conditional Split
- Destinations – SQL Server tables, data warehouses, files

5. SSIS in the Data Engineering Lifecycle

- SSIS is commonly used to:
- Build enterprise ETL pipelines
- Populate data warehouses and reporting databases
- Support BI tools such as Power BI and SSRS
- Integrate operational systems with analytics platforms

Transition to Practice :

- After understanding SQL fundamentals, learners can use SSIS to:
- Automate SQL queries
- Move data across systems
- Clean and transform datasets
- Build end-to-end ETL solutions



Basic SSIS Concepts (Foundational Level)

1. What is SSIS?

SQL Server Integration Services (SSIS) is Microsoft's enterprise ETL (Extract, Transform, Load) platform used to move, transform, and integrate data across multiple systems. At a foundational level, SSIS enables users to automate data movement and perform basic transformations in a reliable and structured way, forming the backbone of many data engineering workflows.

2. Why Use SSIS?

At the basic level, SSIS is used to extract data from common sources such as relational databases, flat files, and spreadsheets, apply simple data cleansing and transformation rules, and load the results into reporting or analytical databases. It supports automation of repetitive data tasks and provides built-in mechanisms for handling errors and basic logging.

3. Core SSIS Architecture (Basics)

The foundational architecture of SSIS consists of the Control Flow, which defines the order in which tasks execute, and the Data Flow, which manages how data is extracted, transformed, and loaded. Connection Managers define how SSIS connects to source and destination systems, while variables and parameters allow packages to be flexible and reusable.

4. Basic Data Flow Components

At an introductory level, learners work with common SSIS data flow components. These include sources such as SQL Server, Flat Files, Excel, and OLE DB, basic transformations like Derived Column and Conditional Split, and destinations such as SQL Server tables or flat files. These components enable simple yet effective data movement and transformation.

Advanced SSIS Concepts (Data Engineering Level).

5. Advanced Data Transformation and Enrichment

At an advanced level, SSIS is used to perform complex transformations such as data enrichment using Lookup and Merge transformations, handling slowly changing dimensions, and managing complex business rules. These capabilities allow SSIS to support enterprise-grade data warehouses and analytics platforms.

6. Automation, Scheduling, and Reusability

Advanced SSIS usage focuses on building dynamic, reusable packages using parameters, expressions, and environment configurations. Packages are deployed to the SSIS Catalog (SSISDB) and scheduled using SQL Server Agent, enabling fully automated, production-ready ETL workflows.

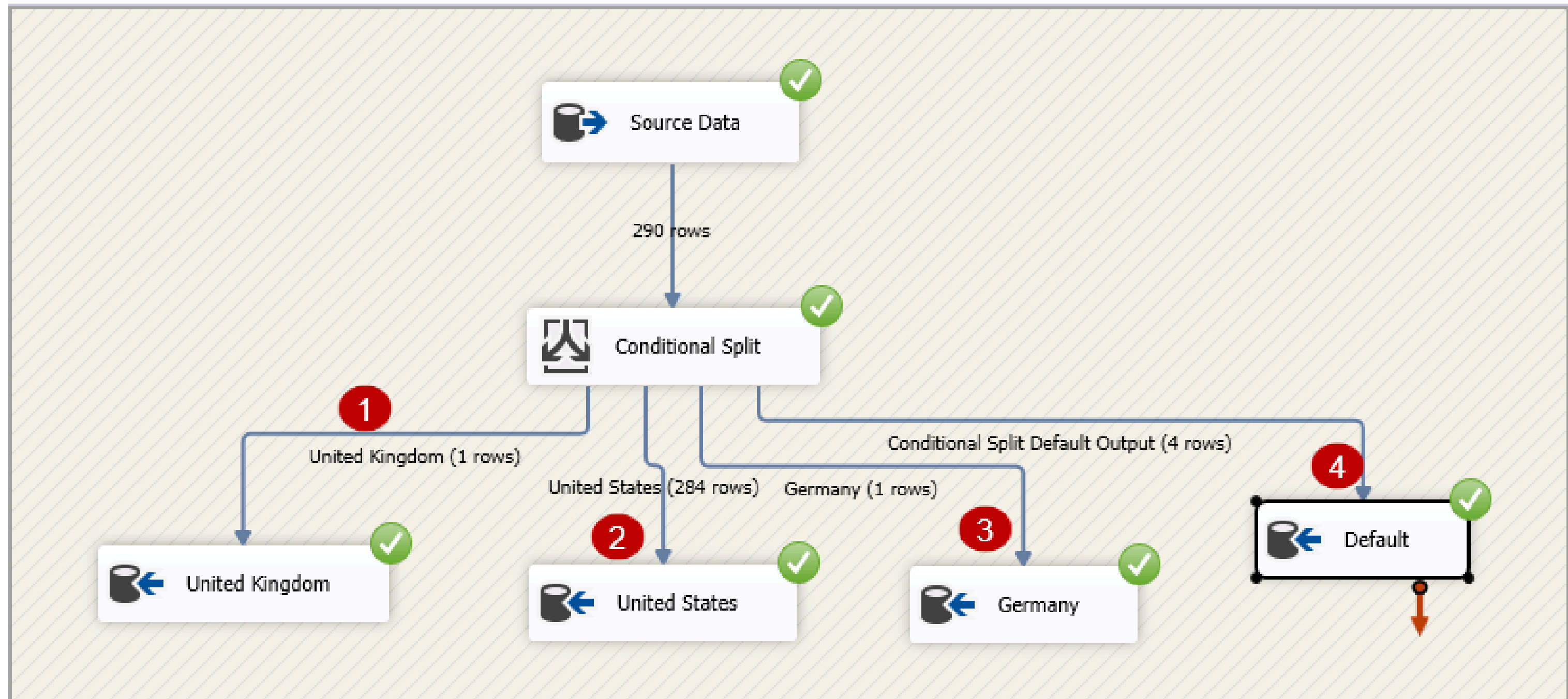
7. Error Handling, Logging, and Monitoring

In enterprise environments, SSIS provides advanced error handling through event handlers, row-level error outputs, and detailed logging. These features allow Data Engineers to monitor pipeline health, quickly diagnose failures, and ensure reliable data delivery.

8. SSIS in the Data Engineering Lifecycle

At scale, SSIS plays a key role in the data engineering lifecycle by powering enterprise ETL pipelines, populating data warehouses, and integrating operational systems with analytics platforms. SSIS often works alongside BI tools such as Power BI and SSRS, ensuring clean, structured, and timely data for reporting and decision-making.

Advanced SSIS Concepts (Data Engineering Level).



EXAMPLE OF PYTHON BASED ETL

```
main.py > ...
1  import pandas as pd
2  import sqlite3
3  import logging
4
5  # -----
6  # CONFIG
7  # -----
8  INPUT_CSV = "input.csv"
9  DATABASE = "etl_data.db"
10 TABLE_NAME = "clean_data"
11
12 # -----
13 # LOGGING
14 # -----
15 logging.basicConfig(
16     level=logging.INFO,
17     format="%(asctime)s - %(levelname)s - %(message)s"
18 )
19
20 # -----
21 # EXTRACT
22 # -----
23 def extract():
24     logging.info("Extracting data from CSV...")
25     return pd.read_csv(INPUT_CSV)

```

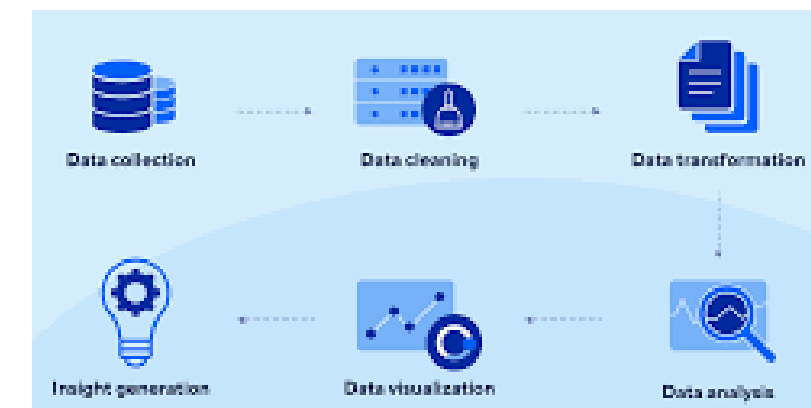
```
# -----
# ETL PIPELINE
# -----
def run_etl():
    try:
        df = extract()
        df = transform(df)
        load(df)
        logging.info("ETL Pipeline completed successfully!")

    except Exception as e:
        logging.error(f"ETL Pipeline failed: {e}")

# -----
# MAIN
# -----
if __name__ == "__main__":
    run_etl()

```

Python ETL Pipeline Processing (Basic Level)



1. What is Python ETL?

Python ETL refers to the use of Python programming to extract data from various sources, transform it into clean and structured formats, and load it into databases or analytical systems. Python is widely used in data engineering due to its simplicity, flexibility, and rich ecosystem of data processing libraries.

2. Why Use Python for ETL?

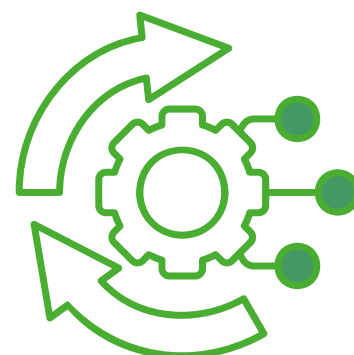
At a foundational level, Python is used for building lightweight, customizable ETL pipelines. It allows Data Engineers to automate data workflows, process files and API data, and apply transformation logic that may be difficult to implement using graphical tools alone. Python is especially useful for scripting, automation, and integrating diverse data sources.

3. Data Extraction in Python

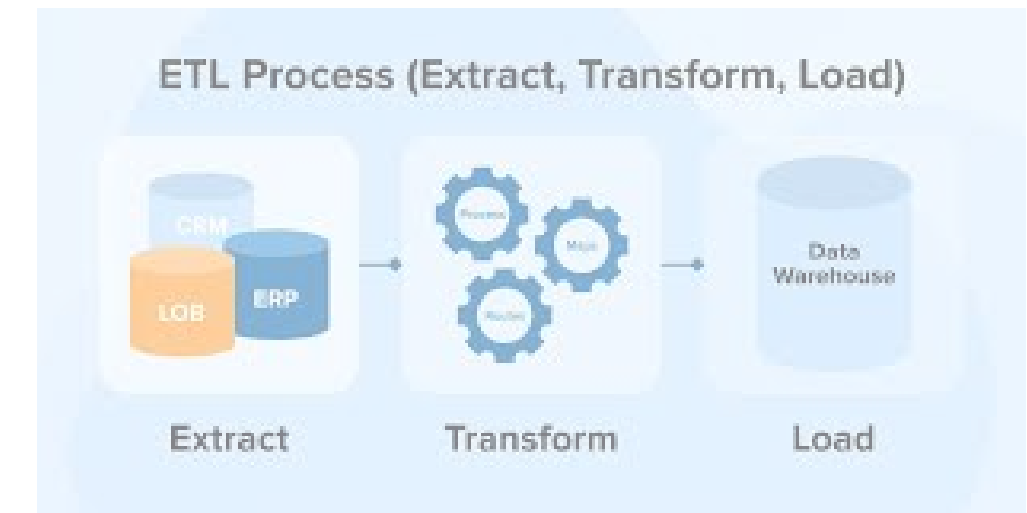
Data extraction involves collecting data from different sources such as CSV and Excel files, relational databases, and APIs. Using libraries like pandas, requests, and database connectors, Python enables controlled and repeatable data ingestion processes suitable for small to medium-scale workloads.

4. Data Transformation in Python

During the transformation stage, raw data is cleaned, filtered, and reshaped into usable formats. Common tasks include handling missing values, converting data types, renaming columns, and applying business rules. Libraries such as pandas and numpy are commonly used to perform these operations efficiently.



Python ETL Pipeline Processing (Basic Level)



5. Data Loading in Python

The loading phase involves writing transformed data into target systems such as SQL databases, data warehouses, or flat files. Python supports database loading through SQL connectors and ORM tools, enabling seamless integration with relational databases used in analytics and reporting.



6. Basic Automation and Error Handling

At the basic level, Python ETL pipelines include simple automation using scripts and scheduled tasks, along with basic error handling using logging and exception management. These practices ensure reliability and repeatability of data processing tasks.



Capstone Overview (Data Engineering)

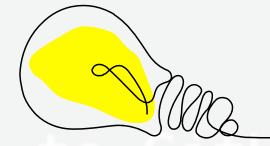
For this capstone project, we will design and implement an end-to-end data engineering pipeline using SQL Server Integration Services (SSIS) as the core ETL tool. The project will focus on ingesting, transforming, and loading structured data into SQL Server databases through well-designed SSIS packages.

All project assets, including SSIS packages, SQL scripts, configuration files, and documentation, will be version-controlled and maintained in GitHub. The repository will demonstrate a clean and logical structure, meaningful commit history, and best practices for collaborative development.

GitHub will be used to showcase SSIS data engineering best practices, including modular package design, parameterization, error handling, logging, and deployment readiness. Comprehensive documentation will be provided through clear README files explaining the end-to-end pipeline architecture, data flow, and execution steps.

BrightLearn will assess the GitHub repository based on code and package quality, project organization, pipeline reliability, documentation clarity, and adherence to industry best practices for SSIS-based data engineering solutions.





Introduction to Github

1. What is GitHub?

GitHub is a web-based platform used for version control and collaboration. It allows developers to store, manage, and track changes to their code using Git. GitHub also makes it easy to collaborate with others, contribute to open-source projects, and showcase your work.

2. Why Create a GitHub Account?

Creating a GitHub account enables you to host code repositories, collaborate with other developers, and manage software projects efficiently. It is widely used by developers, data engineers, and IT professionals to share code, track issues, and build professional portfolios.

3. Sign Up for a GitHub Account

To create a GitHub account, visit <https://github.com> and click Sign up. Enter your email address, create a username and password, and follow the on-screen instructions to verify your account. GitHub may ask a few setup questions to personalize your experience.

4. Set Up and Customize Your Profile

Once your account is created, you can set up your profile by adding a profile picture, bio, and other relevant details. Customizing your profile helps others understand your skills and interests and makes your account more professional and recognizable.



How BrightLearn Will Assess Your GitHub

Data Engineering Portfolio Evaluation Criteria :

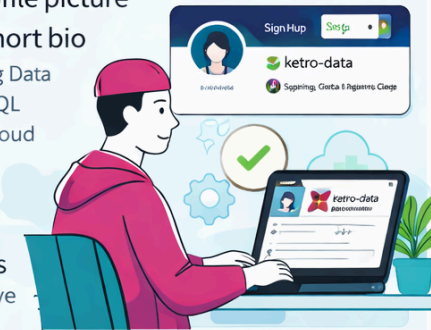
BrightLearn evaluates Data Engineering portfolios using a structured rubric focused on technical depth, engineering practices, and professional presentation.

Technical Engineering Skills

- Complexity and correctness of SQL used for data extraction, transformation, and loading (joins, aggregations, filters, CASE statements)
- Quality of Python-based data processing, including data cleaning, transformation logic, and basic pipeline structuring
- Design and implementation of ETL/ELT pipelines (SSIS, Python scripts, or similar tools)
- Reliability, performance considerations, and reusability of data workflows

Make a GitHub Account

1. Go to github.com
2. Click Sign up
3. Use a **professional username** (e.g., ketro-data, ketro-sithole)
4. Use your email address
5. Complete verification and log in
6. Create a **public profile**: ✓
 - Add a profile picture
 - Write a short bio (e.g., Aspiring Data Engineer | SQL | Python | Cloud)
7. Pin your best repositories once you have projects



How BrightLearn Will Assess Your GitHub

BrightLearn will review your GitHub to evaluate your technical readiness and learning progress, not perfection.

Repository Structure

- Clear folder structure
- Meaningful repository names
- Logical organization of files

Code Quality

- Clean, readable code
- Proper naming conventions
- Comments where necessary
- Use of best practices (SQL, Python, data pipelines)

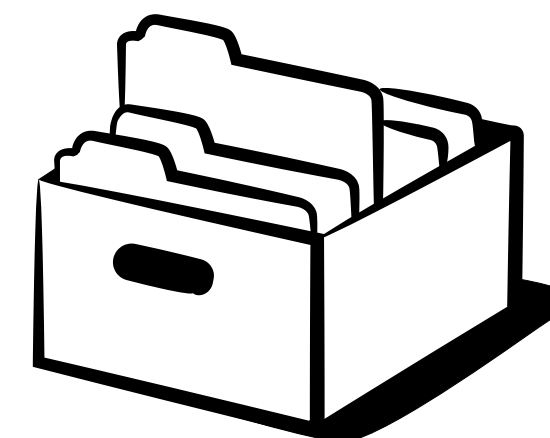




How BrightLearn Will Assess Your GitHub

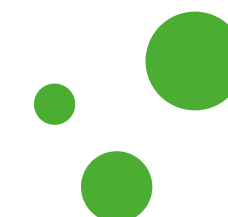
Documentation & Engineering Practices

- Clear explanations of pipeline design, data flow, and architectural decisions
- Inclusion of screenshots or diagrams illustrating pipeline execution and results
- Proper project and folder organization following data engineering best practices
- Professional, concise technical writing and use of README files



Monitoring & Visual Presentation

- Clear representation of pipeline outputs, logs, and validation results
- Well-structured dashboards or reports that consume engineered datasets
- Logical flow from raw data ingestion to final analytics-ready outputs



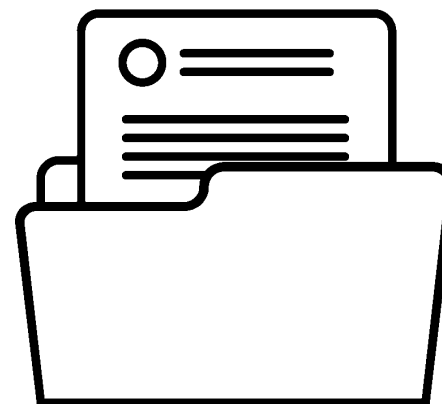
How BrightLearn Will Assess Your GitHub

Tips for a Strong Data Engineering Portfolio

- Start early and iterate continuously on your projects
- Use clean, modular, and well-documented code
- Include architecture diagrams, pipeline screenshots, and execution logs
- Focus on data reliability, scalability, and automation—not just outputs
- Demonstrate version control and professional GitHub practices
- Maintain a polished GitHub profile and link it in your CV and LinkedIn
- Use tools like Canva to create clean project covers and diagrams



A strong data engineering portfolio is not defined by dataset size, but by clarity of design, robustness of pipelines, and the ability to move data reliably from source to insight. Employers look for engineers who can build dependable data systems, and your portfolio is your opportunity to demonstrate that capability.



Thank You



- +27 78 737 2893
- info@brightlearn.co.za
- www.brightlearn.co.za